

Data Cleaning Process

1. Understand the data

```
In [1]: import pandas as pd
import numpy as np

#Load the dataset(Netflix Movies and TV Shows)
df = pd.read_csv(r"C:\Users\Nandhitha\Downloads\archive (2)\New folder\netflix_t
```

```
In [2]: print(df.shape)
```

(8809, 26)

```
In [3]: print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8809 entries, 0 to 8808
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   show_id               8809 non-null  object
1   type                 8809 non-null  object
2   title                8809 non-null  object
3   director             6175 non-null  object
4   cast                 7984 non-null  object
5   country              7978 non-null  object
6   date_added           8799 non-null  object
7   release_year         8809 non-null  int64
8   rating               8805 non-null  object
9   duration             8806 non-null  object
10  listed_in            8809 non-null  object
11  description           8809 non-null  object
12  Unnamed: 12           0 non-null     float64
13  Unnamed: 13           0 non-null     float64
14  Unnamed: 14           0 non-null     float64
15  Unnamed: 15           0 non-null     float64
16  Unnamed: 16           0 non-null     float64
17  Unnamed: 17           0 non-null     float64
18  Unnamed: 18           0 non-null     float64
19  Unnamed: 19           0 non-null     float64
20  Unnamed: 20           0 non-null     float64
21  Unnamed: 21           0 non-null     float64
22  Unnamed: 22           0 non-null     float64
23  Unnamed: 23           0 non-null     float64
24  Unnamed: 24           0 non-null     float64
25  Unnamed: 25           0 non-null     float64
dtypes: float64(14), int64(1), object(11)
memory usage: 1.7+ MB
None
```

```
In [4]: print(df.head())
```

	show_id	type	title	director	cast	country
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa
2	s3	TV Show	Ganglands	Julien Leclercq	NaN	NaN
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India

	date_added	release_year	rating	duration	...	Unnamed: 16
0	September 25, 2021	2020	PG-13	90 min	...	NaN
1	September 24, 2021	2021	TV-MA	2 Seasons	...	NaN
2	September 24, 2021	2021	TV-MA	1 Season	...	NaN
3	September 24, 2021	2021	TV-MA	1 Season	...	NaN
4	September 24, 2021	2021	TV-MA	2 Seasons	...	NaN

	Unnamed: 17	Unnamed: 18	Unnamed: 19	Unnamed: 20	Unnamed: 21
0	NaN	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN

	Unnamed: 22	Unnamed: 23	Unnamed: 24	Unnamed: 25
0	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN

[5 rows x 26 columns]

In [6]: `print(df.describe())`

	release_year	Unnamed: 12	Unnamed: 13	Unnamed: 14	Unnamed: 15	\
count	8809.000000	0.0	0.0	0.0	0.0	
mean	2014.181292	NaN	NaN	NaN	NaN	
std	8.818932	NaN	NaN	NaN	NaN	
min	1925.000000	NaN	NaN	NaN	NaN	
25%	2013.000000	NaN	NaN	NaN	NaN	
50%	2017.000000	NaN	NaN	NaN	NaN	
75%	2019.000000	NaN	NaN	NaN	NaN	
max	2024.000000	NaN	NaN	NaN	NaN	

	Unnamed: 16	Unnamed: 17	Unnamed: 18	Unnamed: 19	Unnamed: 20	\
count	0.0	0.0	0.0	0.0	0.0	
mean	NaN	NaN	NaN	NaN	NaN	
std	NaN	NaN	NaN	NaN	NaN	
min	NaN	NaN	NaN	NaN	NaN	
25%	NaN	NaN	NaN	NaN	NaN	
50%	NaN	NaN	NaN	NaN	NaN	
75%	NaN	NaN	NaN	NaN	NaN	
max	NaN	NaN	NaN	NaN	NaN	

	Unnamed: 21	Unnamed: 22	Unnamed: 23	Unnamed: 24	Unnamed: 25
count	0.0	0.0	0.0	0.0	0.0
mean	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN

```
In [5]: # drop the junk columns(unnamed 12-unnamed 25)
df=df.drop(columns=[col for col in df.columns if 'Unnamed' in col])
```

```
In [6]: print(df.shape)
```

```
(8809, 12)
```

```
In [7]: print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8809 entries, 0 to 8808
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   show_id         8809 non-null   object
1   type            8809 non-null   object
2   title           8809 non-null   object
3   director        6175 non-null   object
4   cast            7984 non-null   object
5   country         7978 non-null   object
6   date_added      8799 non-null   object
7   release_year    8809 non-null   int64
8   rating          8805 non-null   object
9   duration        8806 non-null   object
10  listed_in       8809 non-null   object
11  description      8809 non-null   object
dtypes: int64(1), object(11)
memory usage: 826.0+ KB
None
```

```
In [8]: df.isnull().sum()
```

```
Out[8]: show_id      0
        type        0
        title       0
        director    2634
        cast        825
        country     831
        date_added  10
        release_year 0
        rating      4
        duration    3
        listed_in   0
        description 0
        dtype: int64
```

```
In [9]: df.isnull().mean()
```

```
Out[9]: show_id      0.000000
        type        0.000000
        title       0.000000
        director    0.299012
        cast        0.093654
        country     0.094335
        date_added  0.001135
        release_year 0.000000
        rating      0.000454
        duration    0.000341
        listed_in   0.000000
        description 0.000000
        dtype: float64
```

Handle the missing values

```
In [10]: df['director']=df['director'].fillna('Unknown')
         df['cast']=df['cast'].fillna('Unknown')
         df['country']=df['country'].fillna('Unknown')
```

The director, cast and country colcumn contain more % of missing value. So, if i drop the rows, nearly a third of the entire dataset will be gone. Therefore, we are using fillna(), to fill the missing values with 'Unknown'.

```
In [11]: df.isnull().sum()
```

```
Out[11]: show_id      0
        type        0
        title       0
        director     0
        cast        0
        country     0
        date_added  10
        release_year 0
        rating      4
        duration    3
        listed_in   0
        description 0
        dtype: int64
```

```
In [12]: df=df.dropna(subset=['date_added','rating','duration'])
```

Why drop rows for date_added, rating, duration because the missing values are in less %, these columns are harder to meaningfully fill. Date_added - you can't guess when Netflix added a show, rating - filling with unknown is fine, but since so few rows are affected, dropping them is simple and cost nothing. Duration - Can't guess the runtime.

```
In [13]: df.isnull().sum()
```

```
Out[13]: show_id      0
         type        0
         title       0
         director    0
         cast        0
         country     0
         date_added  0
         release_year 0
         rating      0
         duration    0
         listed_in   0
         description 0
         dtype: int64
```

```
In [14]: df.dtypes
```

```
Out[14]: show_id      object
         type        object
         title       object
         director    object
         cast        object
         country     object
         date_added  object
         release_year int64
         rating      object
         duration    object
         listed_in   object
         description object
         dtype: object
```

Fix the Data type

```
In [15]: df['date_added'] = pd.to_datetime(df['date_added'].str.strip())
```

```
In [16]: df['year_added'] = df['date_added'].dt.year
         df['month_added'] = df['date_added'].dt.month
```

```
In [17]: df['date_added'].dtype
```

```
Out[17]: dtype('<M8[ns]')
```

```
In [18]: df.dtypes
```

```
Out[18]: show_id      object
         type        object
         title       object
         director    object
         cast        object
         country     object
         date_added  datetime64[ns]
         release_year int64
         rating      object
         duration    object
         listed_in   object
         description  object
         year_added  int32
         month_added int32
         dtype: object
```

Remove duplicates

```
In [19]: # check the exact duplicate rows
         print(df.duplicated().sum())
```

0

There is no identical rows (ie) every rows have unique value , but it doesn't mean there is no duplicate. let's check with more targeted subset

```
In [20]: print(df.duplicated(subset=['title','type','release_year']).sum())
```

3

Why title, type, relase_year together, these three columns act like unique identifier for a show

```
In [21]: print(df.shape)
```

(8792, 14)

```
In [22]: df=df.drop_duplicates(subset=['title','type','release_year'])
```

```
In [23]: print(df.shape)
```

(8789, 14)

```
In [25]: df['country'].value_counts()
```

```
Out[25]: country
United States      2810
India              971
Unknown            828
United Kingdom     418
Japan              243
...
Ireland, Canada, United States, United Kingdom    1
Germany, Canada, United States                   1
Denmark, United Kingdom, South Africa, Sweden, Belgium  1
Serbia, South Korea, Slovenia                    1
United Kingdom, Spain, Belgium                   1
Name: count, Length: 749, dtype: int64
```

Standardize text columns

```
In [27]: df['country_list']=df['country'].str.split(', ')
```

```
In [29]: df['country_list'].head(10)
```

```
Out[29]: 0          [United States]
1          [South Africa]
2          [Unknown]
3          [Unknown]
4          [India]
5          [Unknown]
6          [Unknown]
7  [United States, Ghana, Burkina Faso, United Ki...
8          [United Kingdom]
9          [United States]
Name: country_list, dtype: object
```

```
In [30]: df[['country','country_list']].head(10)
```

```
Out[30]:
```

	country	country_list
0	United States	[United States]
1	South Africa	[South Africa]
2	Unknown	[Unknown]
3	Unknown	[Unknown]
4	India	[India]
5	Unknown	[Unknown]
6	Unknown	[Unknown]
7	United States, Ghana, Burkina Faso, United Kin...	[United States, Ghana, Burkina Faso, United Ki...
8	United Kingdom	[United Kingdom]
9	United States	[United States]

```
In [32]: df['listed_in'].value_counts()
```

```
Out[32]: listed_in
Dramas, International Movies      362
Documentaries                    359
Stand-Up Comedy                  334
Comedies, Dramas, International Movies  274
Dramas, Independent Movies, International Movies  252
...
British TV Shows, TV Dramas, TV Sci-Fi & Fantasy      1
Comedies, Cult Movies, Sports Movies                  1
Action & Adventure, Faith & Spirituality, Sci-Fi & Fantasy  1
Sci-fi, Horror, Action                                1
Drama, Romance, Thriller                              1
Name: count, Length: 515, dtype: int64
```

```
In [37]: df['genres']=df['listed_in'].str.split(', ')
```

```
In [38]: df['genres'].head(10)
```

```
Out[38]: 0      [Documentaries]
1  [International TV Shows, TV Dramas, TV Mysteries]
2  [Crime TV Shows, International TV Shows, TV Ac...
3      [Docuseries, Reality TV]
4  [International TV Shows, Romantic TV Shows, TV...
5      [TV Dramas, TV Horror, TV Mysteries]
6      [Children & Family Movies]
7  [Dramas, Independent Movies, International Mov...
8      [British TV Shows, Reality TV]
9      [Comedies, Dramas]
Name: genres, dtype: object
```

```
In [42]: df['duration_num']=df['duration'].str.extract(r'(\d+)').astype(float)
df['duration_unit']=df['duration'].str.extract(r'([A-Za-z]+)')
```

```
In [43]: df[['type', 'duration', 'duration_num', 'duration_unit']].head(10)
```

```
Out[43]:
```

	type	duration	duration_num	duration_unit
0	Movie	90 min	90.0	min
1	TV Show	2 Seasons	2.0	Seasons
2	TV Show	1 Season	1.0	Season
3	TV Show	1 Season	1.0	Season
4	TV Show	2 Seasons	2.0	Seasons
5	TV Show	1 Season	1.0	Season
6	Movie	91 min	91.0	min
7	Movie	125 min	125.0	min
8	TV Show	9 Seasons	9.0	Seasons
9	Movie	104 min	104.0	min

Final Validation

```
In [45]: assert(df['release_year']<=2026).all()
assert df['type'].isin(['Movie','TV Show']).all()
print(df[['duration_num','duration_unit']].isnull().sum())
```

```
duration_num    0
duration_unit    0
dtype: int64
```

```
In [46]: print("Final Shape:",df.shape)
print(df.dtypes)
```

```
Final Shape: (8789, 18)
show_id      object
type         object
title        object
director     object
cast         object
country      object
date_added   datetime64[ns]
release_year  int64
rating       object
duration     object
listed_in    object
description  object
year_added   int32
month_added  int32
country_list object
genres       object
duration_num float64
duration_unit object
dtype: object
```

```
In [48]: df.to_csv('netflix_cleaned.csv', index=False)
```

```
In [ ]:
```